

Memory-efficient Continual Learning with Prototypical Exemplar Condensation

M.-Duong Nguyen^{1,3}, Thien-Thanh Dao⁴, Le-Tuan Nguyen^{1,3}, Dung D. Le^{1,2,3}, Kok-Seng Wong^{1,2}

College of Engineering and Computer Science, VinUniversity¹, Hanoi, Vietnam

Center for Environmental Intelligence, VinUniversity², Hanoi, Vietnam

Center for AI Research, VinUniversity³, Hanoi, Vietnam

Faculty of Information Systems, Phenikaa University⁴, Hanoi, Vietnam

duong.nm2@vinuni.edu.vn, thanh.daothien@phenikaa-uni.edu.vn,

{tuan.nl, dung.ld, wong.ks}@vinuni.edu.vn

Abstract

Rehearsal-based continual learning (CL) mitigates catastrophic forgetting by maintaining a subset of samples from previous tasks for replay. Existing studies primarily focus on optimizing memory storage through coreset selection strategies. While these methods are effective, they typically require storing a substantial number of samples per class (SPC), often exceeding 20, to maintain satisfactory performance. In this work, we propose to further compress the memory footprint by synthesizing and storing prototypical exemplars, which can form representative prototypes when passed through the feature extractor. Owing to their representative nature, these exemplars enable the model to retain previous knowledge using only a small number of samples while preserving privacy. Moreover, we introduce a perturbation-based augmentation mechanism that generates synthetic variants of previous data during training, thereby enhancing CL performance. Extensive evaluations on widely used benchmark datasets and settings demonstrate that the proposed algorithm achieves superior performance compared to existing baselines, particularly in scenarios involving large-scale datasets and a high number of tasks.

1. Introduction

In recent years, deep learning has advanced significantly, driving the need for continual learning (CL). In CL, data arrive sequentially over time, requiring deep models to learn and adapt dynamically to evolving information. However, this continual adaptation introduces a critical challenge known as catastrophic forgetting [12], wherein the integration of new knowledge interferes with previously acquired knowledge, often leading to substantial performance degradation. While addressing catastrophic forget-

ting is already challenging, the problem becomes even more complex in real-world applications, such as autonomous systems [31, 50], edge devices [11, 29], and personalized services [8], where models must operate on resource-constrained devices with limited computational power and memory. Furthermore, as the number of tasks continues to grow [58], the demand for efficient and scalable CL methods becomes critical. These constraints highlight the need for approaches that can both mitigate catastrophic forgetting and maintain performance at scale, even under strict resource limitations.

Various CL methods have been proposed to mitigate catastrophic forgetting [1–3, 22, 40, 45, 49, 51, 53, 54, 60]. One widely adopted strategy to mitigate **catastrophic forgetting** is **rehearsal-based approaches** [13, 21, 42, 44, 56]. Fundamental, the rehearsal-based approaches can be categorized into three sub-categories, i.e., experience replay, generative replay, and gradient episodic memory. Experience replay retains a small subset of samples from previously learned tasks and reuses them during training to alleviate catastrophic forgetting. Generative replay [37] replaces stored samples with a generative model that synthesizes pseudo-samples of past tasks, thereby reducing memory requirements while still enabling rehearsal. However, generative replay may still inherit the problem of catastrophic forgetting, since the generative model itself must be continually updated. Gradient episodic memory (GEM) [28] constrains gradient updates on new tasks by leveraging stored samples, ensuring that the new learning process does not increase the loss on previously seen tasks. Nevertheless, GEM often requires substantial memory, as the dimensionality of stored gradients is comparable to that of the model parameters, which is typically much higher than the dimensionality of the raw data. In this work, we focus on approaches based on experience replay.

In experience replay methods, it is crucial to maintain a compact yet representative set of exemplars that ef-

fectively capture the underlying class distributions of the dataset. Recent studies have explored coreset selection [1–3, 22, 40, 45, 49, 51, 53, 54, 60], which aim to select a subset of samples that maximizes the retention of task-relevant knowledge. While these approaches enhance the quality of rehearsal data, they generally require storing a relatively large number of exemplars per class (see Section B). In real-world CL scenarios, as the number of tasks grows [11], the corresponding increase in the number of classes leads to significant memory overhead. Consequently, these methods impose substantial memory demands, thereby limiting their scalability in practice.

Acknowledging these limitations, we introduce Prototypical Coreset Condensation (ProtoCore), a prototypical exemplar condensation framework for CL. ProtoCore directly learns a set of exemplars that closely approximate the class prototypes, thereby maintaining consistency with both the current task and the prototypical exemplars from previous tasks. By adopting this direct learning strategy, ProtoCore mitigates the representation shift caused by reliance on a feature extractor (see Appendix B), which is a key contributor to catastrophic forgetting in CL. Our main contributions are as follows:

1. We address the representation shift introduced by relying on a feature extractor by proposing a method to learn synthetic exemplars whose embeddings closely approximate their corresponding class prototypes.
2. To improve the quality of the learned prototypical synthetic exemplars, we adopt a prototypical network as the feature extractor and introduce a prototypical alignment loss. This loss explicitly guides the network to preserve previously learned knowledge while mitigating catastrophic forgetting, relying solely on stored prototypical exemplars instead of the original training data.
3. To maximize memory efficiency with only a few samples per class, we design a surrogate continual learning strategy for training the classifier. This strategy uses a prototype perturbation technique [62] to generate synthetic samples directly from learned prototypes, enabling substantial mitigation of catastrophic forgetting while requiring only a single exemplar per class in memory.
4. We evaluate our proposed methods against recent baselines across a range of challenging datasets. The experimental results demonstrate that our approach achieves performance competitive with state-of-the-art baselines, while offering substantial improvements in memory.
5. We also investigate long task sequences and observe that many existing methods are not effective at mitigating catastrophic forgetting in this regime. ProtoCore demonstrates strong performance under long sequence settings, indicating promising directions for future research.

2. Background & Preliminaries

This section offers a primer on online CL, prototypical networks, and dataset distillation. Particular emphasis is placed on the online formulation of prototypical networks and the configuration of dataset condensation within a CL framework, as these elements are critical to our method exposition in Section 3.

2.1. Online Continual Learning

Consider a sequence of tasks $\mathcal{T} = \{\mathcal{T}_1, \mathcal{T}_2, \mathcal{T}_3, \dots, \mathcal{T}_T\}$, each of which consists of unique data and classes $\mathcal{T}_t = \{(x_{t,i}, y_{t,i})\}_{i=1}^{|\mathcal{T}_t|}$, where T is the total task number and $|\mathcal{T}_t|$ is the number of data according to task $\mathcal{T}_t$. At every task $\mathcal{T}_t$, there are subset $\mathcal{C}_t$ of $|\mathcal{C}_t|$ classes available, and we have $\mathcal{C} = \mathcal{C}_{1:T} = \{\mathcal{C}_1 \cup \mathcal{C}_2 \cup \dots \mathcal{C}_T\}$.

Yet, for the online setting, each data sample inside the task $(x_{t,i}, y_{t,i})$ is passed only once in a non-stationary stream, with no task identity provided. The target of CL is to maximize the overall accuracy of all the tasks.

Replay-based methods maintain a small auxiliary memory $\mathcal{M}_x = \{(\hat{x}_i, \hat{y}_i)\}_{i=1}^{|\mathcal{M}_x|}$ to store images from past tasks, where $|\mathcal{M}_x|$ is the memory size. During each iteration of current task, a batch of samples $\mathcal{B}_m = \{(\hat{x}_{m,i}, \hat{y}_{m,i})\}_{i=1}^{|\mathcal{B}_m|}$ is sampled from the memory randomly or under certain strategies for joint training with current data $\mathcal{B}_t = \{(x_{t,i}, y_{t,i})\}_{i=1}^{|\mathcal{B}_t|}$, where $|\mathcal{B}_m| < |\mathcal{M}_x|$ and $|\mathcal{B}_t|$ are the mini-batch size of the replay data and the current tasks, respectively. We denote $|\cdot|$ as both of the absolute value of a scalar, and the size of a set. The joint training process is formulated as follows:

$$\phi^* = \arg \min_{\phi} \mathcal{L}_{\mathcal{T}}(\theta, \phi; \mathcal{T}_t) + \lambda \mathcal{L}_{\mathcal{M}}(\theta, \phi; \mathcal{M}_x), \quad (1)$$

where θ, ϕ are the parameters of the feature extractor and classifier, respectively. $\mathcal{L}_{\mathcal{T}}(\cdot; \cdot)$ is the standard training objective for the task, and $\mathcal{L}_{\mathcal{M}}(\cdot; \cdot)$ is the objective for the replay, and λ is a coefficient hyper-parameter for the replay objective. By simultaneously viewing the data from the current and past tasks, the model can obtain new knowledge while reduce the catastrophic forgetting for the past. However, when the storage space is restricted, the memory informativeness become critical for effective replay. And we argue that by only employing the original images, the storage space is still under-utilized.

2.2. Prototypical Networks

Prototypical Networks (PNs) [38] are a class of metric-based meta-learning algorithms designed to generalize to new classes with limited labeled data. Instead of directly learning a parametric classifier, PNs learn a metric space in which classification is performed by computing distances between query samples and class prototypes.

Given a set of classes $\mathcal{C}$, each class $c \in \mathcal{C}$ is represented by a prototype vector, computed as the mean embedding of its support set:

$$\mathbf{p}_c = \frac{1}{|S_c|} \sum_{\mathbf{x}_i \in S_c} f_\theta(\mathbf{x}_i), \quad (2)$$

where S_c is the support set for class c , $f_\theta(\cdot)$ is the feature encoder parameterized by θ , and $\mathbf{p}_c \in \mathbb{R}^d$ denotes the class prototype in the embedding space.

For a query sample $\mathbf{x}_q$, classification is performed by computing its distance to each class prototype using a metric function $d(\cdot, \cdot)$, typically the squared Euclidean distance. The probability that $\mathbf{x}_q$ belongs to class c is given by:

$$P(y = c \mid \mathbf{x}_q) = \frac{\exp(-d(f_\theta(\mathbf{x}_q), \mathbf{p}_c))}{\sum_{c' \in \mathcal{C}} \exp(-d(f_\theta(\mathbf{x}_q), \mathbf{p}_{c'}))}. \quad (3)$$

The loss encourages the encoder to produce embeddings close to their class prototype while being far from others.

Relation to Contrastive Learning. Prototypical networks share a strong conceptual connection with contrastive learning, which learns representations by pulling similar samples together and pushing dissimilar samples apart in the embedding space. A typical objective for contrastive learning is the InfoNCE loss [32]:

$$\mathcal{L}_{\text{contrastive}} = -\log \frac{\exp(\text{sim}(f_\theta(\mathbf{x}_i), f_\theta(\mathbf{x}_j))/\tau)}{\sum_k \exp(\text{sim}(f_\theta(\mathbf{x}_i), f_\theta(\mathbf{x}_k))/\tau)}, \quad (4)$$

where $\text{sim}(\cdot, \cdot)$ is a similarity function (e.g., cosine similarity), and τ is a temperature parameter.

From this perspective, the prototypical network loss can be viewed as a class-level contrastive objective. Instead of contrasting individual instance pairs, PNs contrast query samples against class prototypes, which serve as representative anchors for each class:

- **Positive pair:** the query sample and its true class prototype.
- **Negative pairs:** the query sample and all other class prototypes.

This equivalence highlights that prototypical networks can be interpreted as a structured form of contrastive learning, where the prototype acts as an aggregated representation that reduces noise and provides better generalization. This property makes PNs particularly effective in few-shot and online CL scenarios, where data arrive incrementally and efficient representation learning is crucial.

2.3. Dataset Distillation

In the concept of CL, the goal of dataset condensation (DC) is at every task t , we distill a large dataset $\mathcal{T}_t = \{(x_i, y_i)\}_{i=1}^{|\mathcal{T}_t|}$ containing $|\mathcal{T}_t|$ training images $x_i \in \mathbb{R}^d$ and its labels $y_i \in \{1, 2, \dots, |Y|\}$ into a small dataset $\mathcal{S}_t =$

$\{s_i, y_i\}_{i=1}^{|\mathcal{S}_t|}$ [49] with $|\mathcal{S}_t|$ synthetic image $s_i \in \mathbb{R}^d$, where $|\mathcal{S}_t| \ll |\mathcal{T}_t|$ (2-3 orders of magnitude), $|Y|$ represents the number of classes, and $\mathbb{R}^d$ defines a d -dimensional space. We expect a network $\phi_{\theta^\mathcal{T}}$ trained on the large training set $\mathcal{T}$ on the unseen test dataset, that is:

$$\begin{aligned} \mathbb{E}_{x_i \sim P_\mathcal{T}}[\ell(\phi_{\theta^\mathcal{T}}(x_i), y)] &\simeq \mathbb{E}_{x_i \sim P_\mathcal{S}}[\ell(\phi_{\theta^\mathcal{T}}(x_i), y)], \quad (5) \\ \text{s.t. } \theta^\mathcal{T} &= \arg \min_{\theta^\mathcal{T}} \mathcal{L}^\mathcal{T}(\theta^\mathcal{T}) = \frac{1}{\mathcal{N}_\mathcal{T}} \sum_{(x,y) \in \mathcal{T}} \ell(\phi_{\theta^\mathcal{T}}(x_i), y) \\ \theta^\mathcal{S} &= \arg \min_{\theta^\mathcal{S}} \mathcal{L}^\mathcal{S}(\theta^\mathcal{S}) = \frac{1}{\mathcal{N}_\mathcal{S}} \sum_{(x,y) \in \mathcal{S}} \ell(\phi_{\theta^\mathcal{S}}(x_i), y) \end{aligned}$$

where $P_\mathcal{T}$ represents the real distribution of the test dataset, x_i represents the input image, y represents the ground truth, and $\ell(\cdot)$ represents a loss function such as cross-entropy.

3. Method

In this section, we present our proposed **ProtoCore** framework. Overall, ProtoCore comprises three main components: (1) an efficient replay memory that stores prototypical exemplars, (2) prototypical exemplar generation via model inversion, and (3) CL with prototypical exemplars through prototype perturbations. An overview of the framework is illustrated in Figure 1.

3.1. Efficient Replay Memory

To design the efficient replay memory which only contains the prototypical exemplars, in which contain only one sample per class, we design a prototypical exemplar generation which based on the model inversion technique [46]. Specifically, we inverse the knowledge from the model trained on the current task to generate exemplars that serve as prototypes for each class. As a consequence, we can ignore the utilization of the generative models in previous works [24, 62]. It is worth noting that the use of generative models with dimensionality M introduces considerable computational overhead, as backpropagation must be performed through the generative network, in contrast to directly generating exemplars of size $C \times H \times W$ from model inversion.

3.2. Prototypical Exemplar Generation.

Our objectives are threefold. First, the synthetic exemplars should maintain minimal distance to the learned prototypes of the current tasks, ensuring they accurately represent the newly acquired knowledge. Second, they should also remain close to the prototypes of the previous tasks, thereby preserving prior knowledge. Third, we should avoid the representation shift due to the synthetic data generation [26]. Consequently, the joint optimization is as follows:

$$z = \arg \min_z \mathcal{L}_{\text{cur-syn}} + \mathcal{L}_{\text{pre-syn}} + \mathcal{L}_{\text{shift}}. \quad (6)$$

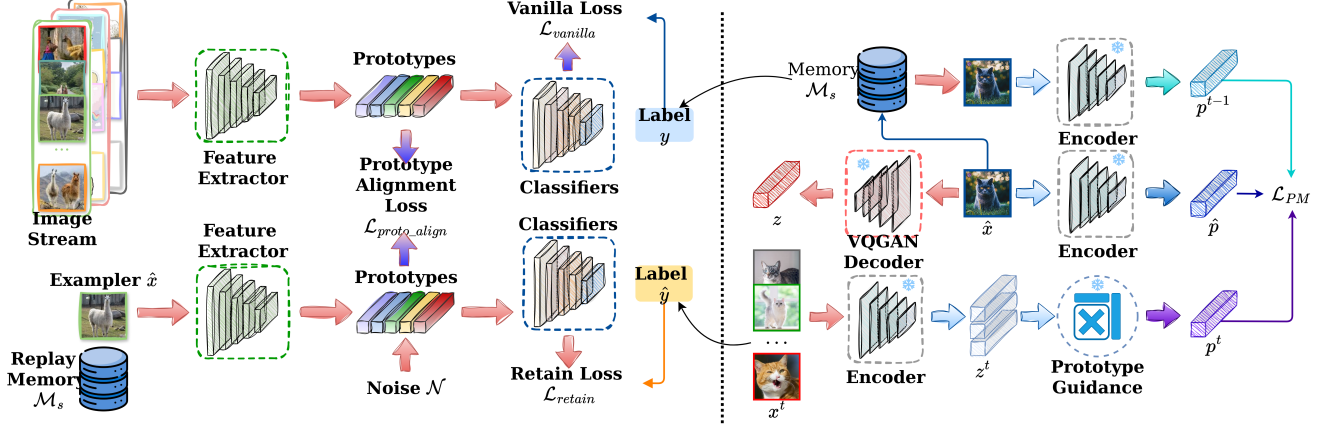


Figure 1. Illustration of ProtoCore architecture.

Given the optimized latent variable z , the corresponding synthetic image is produced by the pretrained decoder as $s = g(z)$. This joint optimization significantly mitigates catastrophic forgetting, as the model is encouraged to selectively retain the most salient and task-relevant features from past tasks while discarding trivial information. As a result, the learned memory remains stable and resistant to forgetting.

3.2.1. Current Exemplar Alignment.

To align the exemplars with the prototypical representations of the current task, the core idea is to synthesize data that produces feature representations closely matching the prototypes of the current task. Given a target class t , the prototypical exemplars can be synthesized using the feature extractor f_θ as follows:

$$\mathcal{L}_{\text{cur_syn}} = \sum_{c \in \mathcal{C}_t} d(f_\theta(s^c); p^c), \quad (7)$$

where $p^c = \frac{1}{|\mathcal{D}_t^c|} \sum_{(x, \cdot) \sim \mathcal{D}_t^c} f_\theta(x)$ denotes the prototype of class c computed from the real data of the current task t . Using the feature extractor, the synthetic exemplar s^c is optimized to form synthetic prototypes that approximate the real ones. The function d measures the distance between the synthetic and real prototypes. In our work, we adopt the mean squared error (MSE) as the distance metric.

3.2.2. Previous Exemplar Alignment.

One distinctive characteristic of ProtoCore is its ability to continuously align the current prototypes with the synthetic prototypes derived from previous tasks. Unlike prior works [24, 27, 62], which perform prototype aggregation through simple or soft averaging between current and previously stored prototypes, our approach focuses on constructing prototypes via exemplar synthesis. This design enables the synthesized exemplars to preserve as much discriminative

information as possible from both the previously stored prototypes and the current task representations. Formally, the alignment loss with respect to the previous task is defined as follows:

$$\mathcal{L}_{\text{pre_syn}} = \sum_{c \in \mathcal{C}_{1:t}} d(f_\theta(s^c); f_\theta(\hat{s}^c)), \quad (8)$$

where $\hat{s}^c$ are the stored synthetic exemplars that capture the information of previous tasks and drawn from $\mathcal{M}_s$.

Discussion on synthesize over misclassified samples.

Since the prototypes are computed from real data, the feature extractor may generate representations in which some samples are not well captured. Consequently, the prototype estimation $p^c = \frac{1}{|\mathcal{D}_t^c|} \sum_{(x, \cdot) \sim \mathcal{D}_t^c} f_\theta(x)$ can become inaccurate when computed using mini-batches. To mitigate this issue, we refine the prototype computation by excluding samples that are misclassified by the classifier. This filtering process allows us to estimate more reliable class prototypes, thereby improving the effectiveness of the prototypical exemplar alignment and leading to better overall performance.

Discussion on Representation Shift. Similar to the phenomenon observed with synthetic data in meta-learning [26], the evolving nature of synthetic datasets can induce shifts in the distribution of prototypical exemplars stored in memory. This effect becomes more pronounced as the number of tasks increases and the synthetic data generation process is repeated multiple times. To maintain consistency in CL when synthetic exemplars are employed, we store the real prototypes $\hat{p}^c$ for each class in memory. These real prototypes serve as anchors that guide the synthetic exemplars, preventing them from drifting excessively from the true representations. Specifically, we introduce the following regularization term:

$$\mathcal{L}_{\text{shift}} = \sum_{c \in \mathcal{C}_{1:t}} d(f_\theta(s^c), \hat{p}^c), \quad (9)$$

Since only one real prototype is retained per class, the additional memory overhead introduced by this mechanism remains minimal.

3.3. Continual Learning with Prototypical Exemplars

In this work, our objective is to develop a memory-efficient CL approach that requires storing only few sample per class. This constraint fundamentally distinguishes our method from existing approaches, as we cannot directly use exemplars from the replay memory to mitigate catastrophic forgetting from previous tasks. The limitation arises because a single sample per class is insufficient to achieve strong class discrimination, potentially leading to imbalanced learning when combined with data from current tasks. To address this challenge, we aim to leverage class prototypes generated from the exemplars and synthesize additional data through controlled perturbations, following the strategy proposed in Zhu et al. [62].

3.3.1. Incremental Feature Extractor

To enhance the quality of synthetic prototypical exemplars, it is essential to construct accurate real prototypes for each task. Consequently, we integrate a prototypical network into the feature extractor. Within the CL framework, the objective of the prototypical network is to learn representations that effectively capture both current and previously learned tasks. To achieve this, we introduce two prototypical loss functions: one for the current task and another for the previously learned tasks. If class c is available in classes of current task, we deploy the prototypical loss on the current data task as follows:

$$\mathcal{L}_{\text{cur.pro}}^c = \sum_{(x^c, y^c) \in \mathcal{D}_t} \frac{\exp(-d(f_\theta(x^c); p^c))}{\sum_{c' \neq c} \exp(-d(f_\theta(x^c); p^{c'}))}, \quad (10)$$

where d represents the statistical loss (e.g., MSE) between two representations. To mitigate the forgetting issue in the feature extractor as tasks progress, we design a prototypical mechanism that aligns the current prototypes with the stored prototypical exemplars. Specifically, when class c is available in the replay memory, we leverage its exemplars to approximate the distribution of previously seen classes and compute the prototypical loss on prior tasks accordingly.

$$\mathcal{L}_{\text{pre.pro}}^c = \frac{\exp(-D(\mathbf{z}^c; p^c))}{\sum_{c' \neq c} \exp(-D(\mathbf{z}^c; p^{c'}))}, \quad (11)$$

where $\mathbf{z}^c = \{f_\theta(h(s^c)) \mid \forall h \in \mathcal{F}\}$ denotes the set of feature representations generated by applying perturbations $h(\cdot)$ from the transformation set $\mathcal{F}$ to the synthetic exemplars s^c of class c .

Discussion on Prototypical Loss. We initially experimented with the prototypical loss [38]. However, the re-

sulting class representations were not well-separated. Well-separated classes are crucial for learning high-quality prototypes, as they enable the prototypes to more accurately represent their respective classes and allow perturbations among prototypes to better capture class-level variations. Consequently, we opted to use contrastive loss instead. Contrastive loss explicitly encourages separation between samples from different classes, leading to more distinct and informative prototypes when computed as the class centers.

3.3.2. Task-head Learning with Memory

To alleviate catastrophic forgetting in CL settings, we apply the task loss to both the current data and the stored exemplars from previous tasks. For the current task, standard task learning is performed using only the classes available in the current dataset. For instance,

$$\mathcal{L}_{\text{task.cur}} = \mathcal{L}_{\text{task}}(f_\phi(z), y), \quad (12)$$

where the latent feature z is obtained from the feature extractor f_θ as $z = f_\theta(x)$ for all $x \in \mathcal{D}_t$. The key distinction between ProtoCore and other CL approaches lies in the use of class-specific prototypical exemplars s^c , which help the model retain knowledge from previous tasks. However, since only a limited number of SPCs are stored, it is often challenging for existing methods to effectively perform rehearsal on past tasks. To address this limitation, our method adopts the prototype augmentation technique proposed by [62] to generate diverse synthetic latent representations for each class c . Specifically,

$$\mathcal{L}_{\text{task.pre}} = \sum_{c \in \mathcal{C}_{1:t}} \mathcal{L}_{\text{task}}^c(f_\phi(\mathbf{z}^c), c), \quad (13)$$

where $\mathbf{z}^c = \{f_\theta(s^c) + \zeta \mid \zeta \sim \mathcal{N}(0, \mathbb{I}\sigma^2)\}$ and ζ denotes Gaussian noise with the same dimensionality as the prototype. By introducing prototype augmentation, we can approximate the underlying data distribution of each class, enabling the model to preserve knowledge of previous tasks even in the absence of real data. The overall task head learning is calculated as:

$$\mathcal{L}_{\text{task}} = \mathcal{L}_{\text{task.cur}} + \mathcal{L}_{\text{task.pre}}. \quad (14)$$

4. Experimental Settings

Baselines: In this work, we compare our proposed ProtoCore with representative baselines from three major categories: efficient memory for CL, replay-based CL, and replay-free CL. These comparisons are designed to address the following key questions: (1) Does ProtoCore provides superior memory efficiency and better mitigation of catastrophic forgetting compared to other memory-based approaches? (2) Does ProtoCore achieve higher performance

Table 1. Last (A_T) and learning (A_L) accuracy (%) comparison on benchmarks with total number of tasks T . Results are averaged over 10 runs with mean $\pm$ standard deviation. **Best** and **Second Best** results are highlighted. Our ProtoCore with 20 + 1 buffer SPCs indicates that the replay buffer is used to store 20 real samples to support the proposed ProtoCore method.

Type	Method	Buffer SPCs	S-CIFAR-100 [15]		S-CIFAR-100 [15]		S-TinyImageNet [25]		S-ImageNet-1K [15]	
			$T=10$		$T=50$		$T=20$		$T=100$	
			A_T	A_L	A_T	A_L	A_T	A_L	A_T	A_L
Efficient Memory	CSReL	20	22.71 $\pm$ 0.54	77.93 $\pm$ 0.27	12.78 $\pm$ 0.69	75.94 $\pm$ 0.33	13.12 $\pm$ 0.82	57.44 $\pm$ 0.90	4.25 $\pm$ 0.48	56.71 $\pm$ 1.21
	BCSR	20	31.52 $\pm$ 0.12	78.47$\pm$0.14	13.42 $\pm$ 0.24	80.64$\pm$0.82	16.84 $\pm$ 0.51	55.09 $\pm$ 1.15	5.05 $\pm$ 0.88	57.85 $\pm$ 1.44
	OCS	20	29.94 $\pm$ 1.10	75.89 $\pm$ 0.91	12.52 $\pm$ 0.66	77.15 $\pm$ 1.72	12.87 $\pm$ 1.14	59.75 $\pm$ 0.77	4.34 $\pm$ 0.34	58.92 $\pm$ 0.84
	PBCS	20	35.95$\pm$1.21	85.34$\pm$2.13	14.57 $\pm$ 0.68	86.90$\pm$0.83	18.12$\pm$0.73	74.63$\pm$1.51	6.01 $\pm$ 0.41	76.45$\pm$2.45
	OnPro	20	29.38 $\pm$ 0.94	77.51 $\pm$ 0.43	12.64 $\pm$ 0.30	75.77 $\pm$ 0.27	14.74 $\pm$ 0.28	56.58 $\pm$ 1.56	4.75 $\pm$ 0.44	58.56 $\pm$ 0.32
Replay based	iCaRL	20	25.92 $\pm$ 0.78	55.52 $\pm$ 0.81	11.25 $\pm$ 0.19	72.52 $\pm$ 0.37	10.56 $\pm$ 0.24	56.58 $\pm$ 0.17	5.05 $\pm$ 0.25	52.13 $\pm$ 0.41
	DER	20	20.44 $\pm$ 0.72	71.50 $\pm$ 1.76	11.63 $\pm$ 0.10	73.57 $\pm$ 0.34	12.91 $\pm$ 1.33	59.24 $\pm$ 1.16	4.45 $\pm$ 0.19	54.93 $\pm$ 0.38
	DER++	20	23.35 $\pm$ 0.22	72.26 $\pm$ 0.47	12.24 $\pm$ 0.21	80.51 $\pm$ 1.53	10.31 $\pm$ 0.52	60.35 $\pm$ 0.74	4.17 $\pm$ 0.62	57.84 $\pm$ 1.28
	C-Flat	20	26.81 $\pm$ 0.94	65.18 $\pm$ 0.56	11.84 $\pm$ 0.38	74.95 $\pm$ 2.05	14.37 $\pm$ 0.68	65.98 $\pm$ 1.88	5.05 $\pm$ 1.17	63.77 $\pm$ 1.54
Replay free	LDC	0	17.54 $\pm$ 0.67	53.82 $\pm$ 0.76	9.12 $\pm$ 0.43	70.12 $\pm$ 1.39	12.31 $\pm$ 0.44	50.17 $\pm$ 1.24	4.82 $\pm$ 0.19	52.46 $\pm$ 0.71
	FeCAM	0	15.89 $\pm$ 0.24	50.63 $\pm$ 0.20	7.26 $\pm$ 0.19	65.02 $\pm$ 1.82	12.03 $\pm$ 0.48	45.98 $\pm$ 1.07	4.34 $\pm$ 0.22	47.91 $\pm$ 1.34
	ADC	0	18.68 $\pm$ 0.14	60.73 $\pm$ 2.10	11.28 $\pm$ 0.58	74.56 $\pm$ 2.72	14.58 $\pm$ 0.58	65.78 $\pm$ 1.52	5.97 $\pm$ 0.41	63.85 $\pm$ 1.48
Ours	ProtoCore	1	18.26 $\pm$ 0.48	75.12 $\pm$ 2.12	12.31 $\pm$ 1.34	77.79 $\pm$ 1.59	14.62 $\pm$ 1.63	66.15 $\pm$ 2.23	6.57 $\pm$ 0.34	64.90 $\pm$ 1.62
	ProtoCore	5	31.17 $\pm$ 0.83	76.85 $\pm$ 1.39	15.29$\pm$0.54	74.98 $\pm$ 2.09	17.31 $\pm$ 0.37	67.15 $\pm$ 1.40	9.03$\pm$0.22	67.12$\pm$1.71
	ProtoCore	20 + 1	37.51$\pm$0.29	76.58 $\pm$ 2.03	24.11$\pm$0.44	76.37 $\pm$ 1.86	27.11$\pm$0.75	68.91$\pm$1.37	14.22$\pm$0.47	65.86 $\pm$ 1.81

than existing rehearsal-based methods? (3) Does ProtoCore yield significant improvements over replay-free CL approaches? For coreset-based methods, we include RM [2], PBCS [59], BCSR [22], OCS [54], and CSReL [40]. For replay-based CL methods, we consider iCaRL [34], DER [7], DER++ [6], and C-Flat [4]. For replay-free CL, we include LDC [17], FeCAM [18], and ADC [19].

Dataset: To evaluate the effectiveness of ProtoCore, particularly under CL settings with long task sequences, we conduct experiments on three standard benchmarks: S-CIFAR-100 [15], S-TinyImageNet [25], and S-ImageNet-1K [15]. We follow the same dataset splits as used in prior works [15, 25] to ensure fair comparison. Each benchmark dataset is divided into multiple subsets with non-overlapping label spaces, where each task contains the same number of classes. Specifically, the total number of classes satisfies $C_t \times T = C$, where C_t denotes the number of classes per task, T the number of tasks, and C the total number of classes in the dataset. Please refer to the supplementary material for details on the dataset configurations.

Models: We use ResNet-18 for S-CIFAR-100, ResNet-50 for S-TinyImageNet, S-ImageNet-1K. Each model is trained with a batch size of 1024 for 200 epochs per task. Please refer to the supplementary material for additional training details on S-CIFAR-100, S-TinyImageNet, and S-ImageNet-1K (see Appendix E). We use VQGAN [] for our

pretrained image decoder.

5. Experimental Evaluation

Main Results. The result comparisons between ProtoCore and three other categories (i.e., efficient-memory, replay-based, and replay-free CL methods) are presented in Table 1. In general, ProtoCore demonstrates strong and balanced performance across both short-task and long-task settings. For short-task scenarios ($T = 10, 20$), ProtoCore achieves competitive or superior results compared to efficient-memory methods such as BCSR and OnPro, while consistently outperforming replay-based and replay-free baselines. This indicates that ProtoCore effectively retains knowledge and adapts quickly even with limited task sequences. For long-task scenarios ($T = 50, 100$), ProtoCore maintains stable performance and shows strong scalability. Despite using only 1–5 stored samples per class, it achieves comparable accuracy to memory-intensive methods like PBCS, and substantially outperforms replay-free approaches (e.g., LDC, FeCAM, ADC). These results highlight ProtoCore’s ability to achieve a balance between accuracy and memory efficiency, demonstrating robustness as the number of tasks grows.

Catastrophic forgetting analysis. Figure 2 illustrates how different methods preserve performance on previously

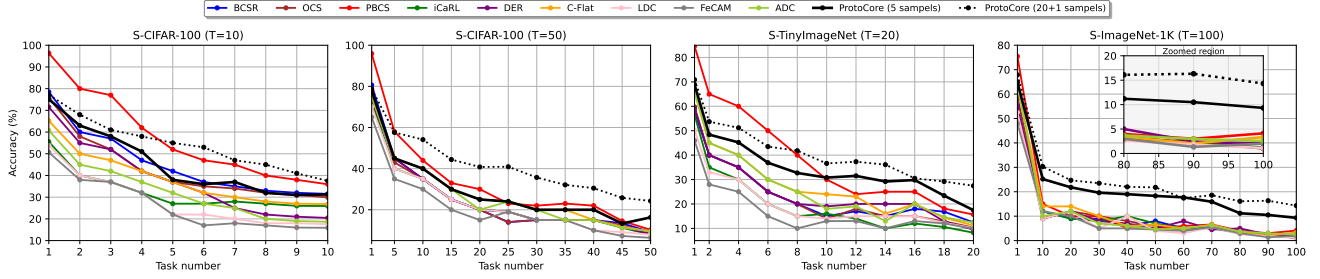


Figure 2. Last accuracy on tasks observed so far in the test set of S-CIFAR-100 (10, 50 tasks), S-TinyImageNet (20 tasks), and S-ImageNet-1K (100 tasks).

learned tasks as new tasks are sequentially introduced. In the short-task settings, i.e., S-CIFAR-100 ($T = 10$) and S-TinyImageNet ($T = 20$), ProtoCore demonstrates competitive resistance to catastrophic forgetting, closely matching the performance of leading efficient-memory methods such as PBCS and BCSR. In contrast, under the long-task settings, i.e., S-CIFAR-100 ($T = 50$) and S-ImageNet-1K ($T = 100$), ProtoCore exhibits stronger resistance to forgetting. While baseline methods experience pronounced accuracy degradation after each new task, ProtoCore maintains a notably more stable performance trajectory throughout the continual learning process.

5.1. Ablation Tests

Table 2. Ablation studies of using different losses for $\mathcal{L}_{\text{pro,cur}}$ and $\mathcal{L}_{\text{pro,pre}}$. The contrastive loss is our proposed baseline, while prototypical loss is the loss using by [38].

Method	S-CIFAR-100 (T=10) Acc $\uparrow$	S-TinyImageNet (T=20) Acc $\uparrow$
Prototypical	29.51 $\pm$ 0.64	15.67 $\pm$ 0.44
Contrastive	31.17 $\pm$ 0.83	17.31 $\pm$ 0.37

Different choice of loss function. Table 2 summarizes the ablation results for different prototypical loss functions. As shown, contrastive learning provides a stronger separation between classes, which is reflected in the improved classification accuracy.

Effect of each component. Table 3 presents the ablation results for each component of the proposed method.

Different number of exemplars. In Figure 4, we conducted a comparative analysis on how number of representative exemplars affects the learning accuracy on S-CIFAR-100 under two different task lengths ($T=10$ and $T=50$). Overall, performance improves consistently as the number

Table 3. Ablation studies on CIFAR-10 and CIFAR-100. ProtoCore denotes the algorithms with all losses. We denote (1) as $\mathcal{L}_{\text{cur,syn}}$, (2) as $\mathcal{L}_{\text{pre,syn}}$, and (3) as $\mathcal{L}_{\text{shift}}$ for synthetic exemplar generation. We denote (4) as $\mathcal{L}_{\text{pro,cur}}$, (5) as $\mathcal{L}_{\text{pro,pre}}$, and (6) as $\mathcal{L}_{\text{task,cur}}$, (7) as $\mathcal{L}_{\text{task,pre}}$.

Method	CIFAR-10 Acc $\uparrow$ (Forget $\downarrow$)	CIFAR-100 Acc $\uparrow$ (Forget $\downarrow$)
(1)	43.12 $\pm$ 0.65(12.5 $\pm$ 0.1)	22.43 $\pm$ 0.40(18.2 $\pm$ 0.5)
(1) + (2)	45.45 $\pm$ 0.83(11.8 $\pm$ 0.3)	25.10 $\pm$ 0.18(16.7 $\pm$ 0.4)
(1) + (2) + (3)	47.60 $\pm$ 0.10(10.9 $\pm$ 0.3)	27.85 $\pm$ 0.86(15.5 $\pm$ 0.8)
(4)	44.20 $\pm$ 0.32(12.2 $\pm$ 0.6)	23.50 $\pm$ 0.39(17.8 $\pm$ 0.4)
(4) + (5)	46.10 $\pm$ 0.41(11.4 $\pm$ 0.3)	26.30 $\pm$ 0.77(16.2 $\pm$ 0.9)
(6)	42.50 $\pm$ 0.34(12.8 $\pm$ 0.9)	21.90 $\pm$ 0.41(18.5 $\pm$ 0.5)
(6) + (7)	44.80 $\pm$ 0.63(12.0 $\pm$ 0.3)	24.60 $\pm$ 0.69(17.0 $\pm$ 0.3)
ProtoCore (ours)	49.25$\pm$0.21(9.8$\pm$0.3)	37.51$\pm$0.93(12.0$\pm$0.7)

of samples increases, with the most substantial gain occurring when moving from 1 to 5 samples. Notably, using 5 samples achieves the highest accuracy for both settings, while further increasing the memory to 7 samples only brings a marginal improvement. Interestingly, the 1-sample setting, despite being the most memory-efficient, already provides a reasonable accuracy baseline and demonstrates strong efficiency compared to larger memory sizes.

Task-wise Synthetic Generation.

Figure 3 presents the t-SNE visualization results of synthetic data generation in the first task on the S-CIFAR-100 test set. Several intuitive observations can be made. (1) Certain classes are easily adapted through synthetic prototypes, requiring only a few optimization rounds

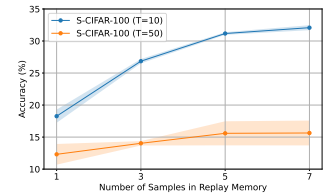


Table 4. Last accuracy of ProtoCore on S-CIFAR-100 with $T=10$ and $T=50$ across different replay memory sizes.

to align with their real counterparts. (2) Misclassified samples can negatively affect the learning of synthetic prototypes. This occurs because, during batch perturbation, misclassified samples may be aggregated together, forming inaccurate real prototypes. Consequently, the optimization process of the synthetic exemplars may be guided in an incorrect direction.

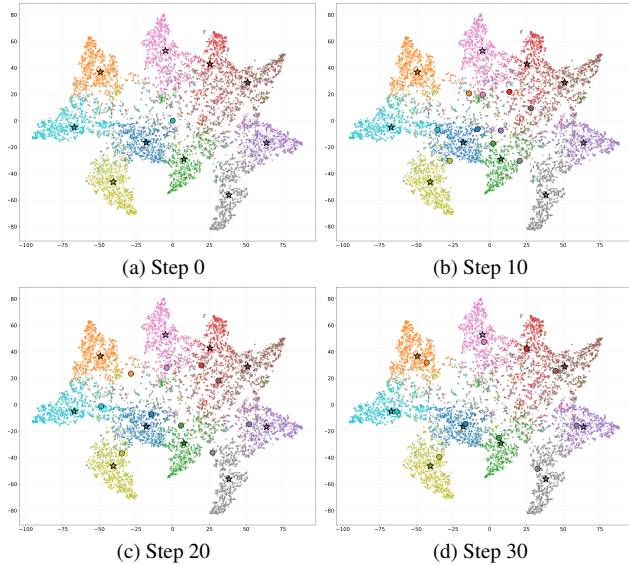


Figure 3. t-SNE visualizations of synthetic features generated by ProtoCore on the CIFAR-100 test set. The synthetic data generation process converges within less than 40 epochs, yielding exemplars that closely approximate the real data distribution. ★ represents the real prototypes, ● represents the embeddings generated by the synthetic exemplars.

Analysis of computational and memory costs. We conducted a comparative analysis of ProtoCore and baseline methods with respect to computational cost (reported as estimated GFLOPs) and memory usage (GPU and maximum HDD usage), as summarized in Table 9. Regarding memory usage, ProtoCore attains the second-lowest footprint among all baselines. This is expected when compared with replay-based methods, as those approaches store a larger number of samples than ProtoCore (20 SPCs compare to 2 from ProtoCore). Although replay-free methods claim not to retain data from previous tasks, methods such as LDC and ADC store full model checkpoints from earlier tasks. These model snapshots require significantly more memory than exemplar storage, leading to considerably higher overall memory consumption. The additional analysis of computational and memory on more challenging data (i.e., S-TinyImageNet) are shown in Appendix F.2

Table 5. Comparison on CIFAR-100 in terms of computation (Hours), GPU memory (GB) and memory usage (MB).

Method	Comp. Time (Hours)	GPU Mem. (GB)	Max HDD Mem. (MB)
CSReL	5.97	1.23	6.20
BCSR	8.16	3.80	6.20
OCS	7.72	3.42	6.20
PBCS	9.79	2.22	6.20
OnPro	8.35	1.75	6.20
iCaRL	0.93	1.31	6.20
DER	1.67	2.10	6.20
DER++	2.10	2.98	6.20
C-Flat	2.51	2.5	6.20
LDC	1.87	7.06	46
FeCAM	1.24	2.15	0
ADC	1.16	6.46	44
ProtoCore	6.58	4.03	0.26

6. Conclusion and Limitations

Conclusion This paper introduces ProtoCore, a framework that synthesizes and stores prototypical exemplars in the experience replay memory as an alternative to conventional data storage. By maintaining only a small number of synthetic samples per class (e.g., 2, 4, or 6), ProtoCore substantially reduces memory requirements while preserving data privacy, since the generated exemplars are not identical to real images. Despite this compression, the framework effectively retains performance in the continual learning setting. Instead of storing raw prototypes, ProtoCore leverages prototypical exemplars that enable the model to update its feature extractor without relying on original training data. Furthermore, an augmentation mechanism applied to these exemplars enhances their diversity, improving the model’s ability to recall and adapt to previous tasks. Extensive experiments on widely used benchmark datasets demonstrate the effectiveness of ProtoCore and its individual components. In future work, we plan to explore more efficient strategies, such as incorporating margin-based objectives to further enhance inter-class discriminability.

Limitations While generating prototypical exemplars demonstrates robustness, a key limitation of both our method and that of [62] lies in the use of noise for data perturbation. Such noise does not reliably preserve the underlying structure of class clusters, as it often fails to form well-defined, spherical distributions. Mitigating this issue will be an important direction for future work. Additionally, achieving high-quality synthetic exemplars for every task is critical for optimal performance in our approach. Developing strategies to balance exemplar fidelity with computational efficiency remains an open challenge.

Acknowledgment

Minh-Duong Nguyen, Le-Tuan Nguyen, and Dung D. Le are with the Center for AI Research (CAIR), VinUniversity. Kok-Seng Wong and Dung D. Le are with the Center for Environmental Intelligence (CEI), VinUniversity. This work was supported in part by the Green Serverless Computing for Resource-Efficient AI Training Project at VinUniversity under Grant VUNI.CEI.FS 0002;

References

- [1] Nader Asadi, Mohammad Reza Davari, Sudhir Mudur, Rahaf Aljundi, and Eugene Belilovsky. Prototype-sample relation distillation: towards replay-free continual learning. In *International conference on machine learning*, pages 1093–1106. PMLR, 2023. 1, 2
- [2] Jihwan Bang, Heesu Kim, YoungJoon Yoo, Jung-Woo Ha, and Jonghyun Choi. Rainbow memory: Continual learning with a memory of diverse samples. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 8218–8227, 2021. 6, 1, 2
- [3] Jihwan Bang, Hyunseo Koh, Seulki Park, Hwanjun Song, Jung-Woo Ha, and Jonghyun Choi. Online continual learning on a contaminated data stream with blurry task boundaries. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 9275–9284, 2022. 1, 2
- [4] Ang Bian, Wei Li, Hangjie Yuan, Mang Wang, Zixiang Zhao, Aojun Lu, Pengliang Ji, Tao Feng, et al. Make continual learning stronger via c-flat. *Advances in Neural Information Processing Systems*, 37:7608–7630, 2024. 6
- [5] Zalán Borsos, Mojmir Mutny, and Andreas Krause. Coresets via bilevel optimization for continual learning and streaming. *Advances in neural information processing systems*, 33: 14879–14890, 2020. 2
- [6] Matteo Boschini, Lorenzo Bonicelli, Pietro Buzzega, Angelo Porrello, and Simone Calderara. Class-incremental continual learning into the extended der-verse. *IEEE transactions on pattern analysis and machine intelligence*, 45(5):5497–5512, 2022. 6, 2
- [7] Pietro Buzzega, Matteo Boschini, Angelo Porrello, Davide Abati, and Simone Calderara. Dark experience for general continual learning: a strong, simple baseline. *Advances in neural information processing systems*, 33:15920–15930, 2020. 6, 2, 8
- [8] Huancheng Chen and Haris Vikalo. Mixed-precision quantization for federated learning on resource-constrained heterogeneous devices. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6138–6148, 2024. 1
- [9] Jinpeng Chen, Runmin Cong, Yuxuan Luo, Horace Ip, and Sam Kwong. Saving 100x storage: Prototype replay for reconstructing training sample distribution in class-incremental semantic segmentation. *Advances in Neural Information Processing Systems*, 36:35988–35999, 2023. 2
- [10] Mingyang Chen, Jiawei Du, Bo Huang, Yi Wang, Xiaobo Zhang, and Wei Wang. Influence-guided diffusion for dataset distillation. In *The Thirteenth International Conference on Learning Representations*, 2025. 1
- [11] Hongrong Cheng, Miao Zhang, and Javen Qinfeng Shi. A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024. 1, 2
- [12] Matthias De Lange and Tinne Tuytelaars. Continual prototype evolution: Learning online from non-stationary data streams. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 8250–8259, 2021. 1, 3
- [13] Junze Deng, Qinhang Wu, Peizhong Ju, Sen Lin, Yingbin Liang, and Ness Shroff. Unlocking the power of rehearsal in continual learning: A theoretical perspective. In *Forty-second International Conference on Machine Learning*, 2025. 1
- [14] Wenxiao Deng, Wenbin Li, Tianyu Ding, Lei Wang, Hongguang Zhang, Kuihua Huang, Jing Huo, and Yang Gao. Exploiting inter-sample and inter-feature relations in dataset distillation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17057–17066, 2024. 2
- [15] Shibhansh Dohare, J Fernando Hernandez-Garcia, Qingfeng Lan, Parash Rahman, A Rupam Mahmood, and Richard S Sutton. Loss of plasticity in deep continual learning. *Nature*, 632(8026):768–774, 2024. 6, 8, 9
- [16] Jiawei Du, Juncheng Hu, Wenxin Huang, Joey Tianyi Zhou, et al. Diversity-driven synthesis: Enhancing dataset distillation through directed weight adjustment. *Advances in neural information processing systems*, 37:119443–119465, 2024. 2
- [17] Alex Gomez-Villa, Dipam Goswami, Kai Wang, Andrew D Bagdanov, Bartłomiej Twardowski, and Joost van de Weijer. Exemplar-free continual representation learning via learnable drift compensation. In *European Conference on Computer Vision*, pages 473–490. Springer, 2024. 6
- [18] Dipam Goswami, Yuyang Liu, Bartłomiej Twardowski, and Joost Van De Weijer. Fecam: Exploiting the heterogeneity of class distributions in exemplar-free continual learning. *Advances in Neural Information Processing Systems*, 36:6582–6595, 2023. 6
- [19] Dipam Goswami, Albin Soutif-Cormerais, Yuyang Liu, Sandesh Kamath, Bart Twardowski, Joost Van De Weijer, et al. Resurrecting old classes with new data for exemplar-free continual learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 28525–28534, 2024. 6
- [20] Jianyang Gu, Saeed Vahidian, Vyacheslav Kungurtsev, Haonan Wang, Wei Jiang, Yang You, and Yiran Chen. Efficient dataset distillation via minimax diffusion. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15793–15803, 2024. 2
- [21] Guy Hacohen and Tinne Tuytelaars. Predicting the susceptibility of examples to catastrophic forgetting. In *Forty-second International Conference on Machine Learning*, 2025. 1

- [22] Jie Hao, Kaiyi Ji, and Mingrui Liu. Bilevel coreset selection in continual learning: A new formulation and algorithm. *Advances in Neural Information Processing Systems*, 36:51026–51049, 2023. 1, 2, 6
- [23] Jiangpeng He and Fengqing Zhu. Exemplar-free online continual learning. In *2022 IEEE International Conference on Image Processing (ICIP)*, pages 541–545, 2022. 2
- [24] Stella Ho, Ming Liu, Lan Du, Longxiang Gao, and Yong Xiang. Prototype-guided memory replay for continual learning. *IEEE transactions on neural networks and learning systems*, 35(8):10973–10983, 2023. 3, 4, 2
- [25] Saihui Hou, Xinyu Pan, Chen Change Loy, Zilei Wang, and Dahua Lin. Learning a unified classifier incrementally via rebalancing. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 831–839, 2019. 6, 8, 9
- [26] Zixuan Hu, Yongxian Wei, Li Shen, Zhenyi Wang, Baoyuan Wu, Chun Yuan, and Dacheng Tao. Task-distributionally robust data-free meta-learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025. 3, 4
- [27] Hongbin Lin, Yifan Zhang, Zhen Qiu, Shuaicheng Niu, Chuang Gan, Yanxia Liu, and Mingkui Tan. Prototype-guided continual adaptation for class-incremental unsupervised domain adaptation. In *European Conference on Computer Vision*, pages 351–368. Springer, 2022. 4, 2
- [28] David Lopez-Paz and Marc’Aurelio Ranzato. Gradient episodic memory for continual learning. *Advances in neural information processing systems*, 30, 2017. 1
- [29] Chengtao Lv, Hong Chen, Jinyang Guo, Yifu Ding, and Xi-anlong Liu. Ptq4sam: Post-training quantization for segment anything. In *Proceedings of the IEEE/CVF Conference on computer vision and pattern recognition*, pages 15941–15951, 2024. 1
- [30] Zhiheng Ma, Anjia Cao, Funing Yang, Yihong Gong, and Xing Wei. Curriculum dataset distillation. *IEEE Transactions on Image Processing*, 2025. 2
- [31] Khai Nguyen, Sam Schoedel, Anoushka Alavilli, Brian Plancher, and Zachary Manchester. Tinympc: Model-predictive control on resource-constrained microcontrollers. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1–7. IEEE, 2024. 1
- [32] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018. 3
- [33] Tian Qin, Zhiwei Deng, and David Alvarez-Melis. A label is worth a thousand images in dataset distillation. *Advances in Neural Information Processing Systems*, 37:131946–131971, 2024. 2
- [34] Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H Lampert. icarl: Incremental classifier and representation learning. In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, pages 2001–2010, 2017. 6
- [35] Matthew Riemer, Ignacio Cases, Robert Ajemian, Miao Liu, Irina Rish, Yuhai Tu, and Gerald Tesauro. Learning to learn without forgetting by maximizing transfer and minimizing interference. In *International Conference on Learning Representations*. 1
- [36] Gobinda Saha, Isha Garg, and Kaushik Roy. Gradient projection memory for continual learning. In *International Conference on Learning Representations*, 2021. 1
- [37] Hanul Shin, Jung Kwon Lee, Jaehong Kim, and Jiwon Kim. Continual learning with deep generative replay. *Advances in neural information processing systems*, 30, 2017. 1
- [38] Jake Snell, Kevin Swersky, and Richard Zemel. Prototypical networks for few-shot learning. *Advances in neural information processing systems*, 30, 2017. 2, 5, 7
- [39] Shixiang Tang, Dapeng Chen, Jinguo Zhu, Shijie Yu, and Wanli Ouyang. Layerwise optimization by gradient decomposition for continual learning. In *Proceedings of the IEEE/CVF conference on Computer Vision and Pattern Recognition*, pages 9634–9643, 2021. 1
- [40] Ruilin Tong, Yuhang Liu, Javen Qinfeng Shi, and Dong Gong. Coreset selection via reducible loss in continual learning. In *The Thirteenth International Conference on Learning Representations*, 2025. 1, 2, 6
- [41] Minh-Tuan Tran, Trung Le, Xuan-May Le, Thanh-Toan Do, and Dinh Phung. Enhancing dataset distillation via non-critical region refinement. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 10015–10024, 2025. 1
- [42] Edoardo Uretini and Antonio Carta. Online curvature-aware replay: Leveraging $\mathbf{2}^{\text{nd}}$ order information for online continual learning. In *Forty-second International Conference on Machine Learning*, 2025. 1
- [43] Kai Wang, Zekai Li, Zhi-Qi Cheng, Samir Khaki, Ahmad Sajedi, Ramakrishna Vedantam, Konstantinos N Plataniotis, Alexander Hauptmann, and Yang You. Emphasizing discriminative features for dataset distillation in complex scenarios. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 30451–30461, 2025. 2
- [44] Xinrui Wang, Shao-Yuan Li, Jiaqiang Zhang, and Songcan Chen. Cut out and replay: A simple yet versatile strategy for multi-label online continual learning. In *Forty-second International Conference on Machine Learning*, 2025. 1
- [45] Yujie Wei, Jiaxin Ye, Zhizhong Huang, Junping Zhang, and Hongming Shan. Online prototype learning for online continual learning. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 18764–18774, 2023. 1, 2, 3
- [46] Yongxian Wei, Zixuan Hu, Li Shen, Zhenyi Wang, Chun Yuan, and Dacheng Tao. Open-vocabulary customization from clip via data-free knowledge distillation. In *The Thirteenth International Conference on Learning Representations*, 2025. 3
- [47] Chenshen Wu, Luis Herranz, Joost Liu, Xialei and previous Van De Weijer, Bogdan Raducanu, et al. Memory replay gans: Learning to generate new categories without forgetting. *Advances in neural information processing systems*, 31, 2018. 1
- [48] Yichen Wu, Long-Kai Huang, Renzhen Wang, Deyu Meng, and Ying Wei. Meta continual learning revisited: Implicitly enhancing online hessian approximation via variance reduction. In *The Twelfth international conference on learning representations*, 2024. 1

- [49] Enneng Yang, Li Shen, Zhenyi Wang, Tongliang Liu, and Guibing Guo. An efficient dataset condensation plugin and its application to continual learning. *Advances in Neural Information Processing Systems*, 36:67625–67642, 2023. 1, 2, 3
- [50] Haohan Yang, Yanxin Zhou, Jingda Wu, Haochen Liu, Lie Yang, and Chen Lv. Human-guided continual learning for personalized decision-making of autonomous driving. *IEEE Transactions on Intelligent Transportation Systems*, 2025. 1
- [51] Fei Ye and Adrian G Bors. Online task-free continual learning via dynamic expansionable memory distribution. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 20512–20522, 2025. 1, 2
- [52] Zeyuan Yin, Eric Xing, and Zhiqiang Shen. Squeeze, recover and relabel: Dataset condensation at imagenet scale from a new perspective. *Advances in Neural Information Processing Systems*, 36:73582–73603, 2023. 1
- [53] Jinsoo Yoo, Yunpeng Liu, Frank Wood, and Geoff Pleiss. Layerwise proximal replay: A proximal point method for online continual learning. In *Forty-first International Conference on Machine Learning*, 2024. 1, 2
- [54] Jaehong Yoon, Divyam Madaan, Eunho Yang, and Sung Ju Hwang. Online coreset selection for rehearsal-based continual learning. In *International Conference on Learning Representations*, 2022. 1, 2, 6
- [55] Yaqian Zhang, Bernhard Pfahringer, Eibe Frank, Albert Bifet, Nick Jin Sean Lim, and Yunzhe Jia. Repeated augmented rehearsal: A simple but strong baseline for online continual learning. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28-December 9, 2022*, 2022. 1
- [56] Bowen Zheng, Da-Wei Zhou, Han-Jia Ye, and De-Chuan Zhan. Multi-layer rehearsal feature augmentation for class-incremental learning. In *Forty-first International Conference on Machine Learning*, 2024. 1
- [57] Xinhao Zhong, Hao Fang, Bin Chen, Xulin Gu, Meikang Qiu, Shuhan Qi, and Shu-Tao Xia. Hierarchical features matter: A deep exploration of progressive parameterization method for dataset distillation. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 30462–30471, 2025. 2
- [58] Da-Wei Zhou, Qi-Wei Wang, Zhi-Hong Qi, Han-Jia Ye, De-Chuan Zhan, and Ziwei Liu. Class-incremental learning: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024. 1
- [59] Xiao Zhou, Renjie Pi, Weizhong Zhang, Yong Lin, Zonghao Chen, and Tong Zhang. Probabilistic bilevel coreset selection. In *International conference on machine learning*, pages 27287–27302. PMLR, 2022. 6, 1
- [60] Yuhao Zhou, Yuxin Tian, Jindi Lv, Mingjia Shi, Yuanxi Li, Qing Ye, Shuhao Zhang, and Jiancheng Lv. Ferret: An efficient online continual learning framework under varying memory constraints. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 4850–4861, 2025. 1, 2
- [61] Dongyao Zhu, Bowen Lei, Jie Zhang, Yanbo Fang, Yiqun Xie, Ruqi Zhang, and Dongkuan Xu. Rethinking data distillation: Do not overlook calibration. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4935–4945, 2023. 1
- [62] Fei Zhu, Xu-Yao Zhang, Chuang Wang, Fei Yin, and Cheng-Lin Liu. Prototype augmentation and self-supervision for incremental learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 5871–5880, 2021. 2, 3, 4, 5, 8